MINISTRY OF INDUSTRY & TRADE
INDUSTRIAL UNIVERSITY OF HO CHI MINH CITY
FACULTY OF INFORMATION TECHNOLOGY

GROUP ASSIGNMENT

Course name: Software Architecture and Design

Course code: 2101452

Department: Software Engineering

Subject leader: Dr Nguyen Trong Tien

Outcome + Rubric

Step 1: Form a group *(Time: first week)*

- You form your groups to do the assignment
- Number of members from 2 to 4 people
- Elect the group leader and assign work to each member

Step 2: Choose a topic to implement *(Time: first week)*

- The group chooses one of the names of the assignments attached below
- Maybe, the group can propose a topic that the group likes, describing the assignment's requirements and she will approve it before you carry it out.

Step 3: Implement the assignment *(Time: week 2 to week 14)*

✓ Implement the assignment consists of the following stages:

1. Collect requirements
2. Analyze requirements
3. Design analysis
4. Implementation
5. Testing
6. Deployment

✓ Output:

1. Source code of the application

Step 4: Present and criticize the group's results *(Time: week 14 or 15, according to the schedule you announce)*

Results are evaluated according to the rubric attached below

Note: If you have any questions, please meet the lecturer directly during theory and practice sessions or you can discuss via the class Zalo Group or the lecturer's email.

CLOs and Rubric

STT	CLO	Rubric (expected)
1	Understand software architecture fundamentals.	Identify and explain the SA fundamentals (at least 3).
2	Understand common software architecture styles.	Present more than one of the common SA styles.
3	Select the appropriate software architectural style.	Present the criteria to choose a specific SA style.
4	Implement a chosen architecture style in software development.	Apply a specific framework to implement primarily parts of the chosen SA style.
5	Modeling a software architecture.	Model most parts of a software architecture in a given context.
6	Present the course project reports.	Present the implemented reports and answer more than 2 question

Some of the Architectures for the assignment

1. Microservice Architecture Style

   Scenario: Building an E-commerce Platform

An e-commerce platform requires several distinct functionalities, including user authentication, product catalog management, order processing, inventory management, payment processing, and recommendation services. Using microservices, each of these functionalities can be developed, deployed, and scaled independently.

■ Example Architecture

1. **User Service**: Handles user registration, authentication, and profile management. This service can use OAuth or JWT for secure access and store user data in a database.
2. **Product Catalog Service**: Manages the catalog of products, including categories, descriptions, prices, and images. This service handles product searches and filters. It could use a NoSQL database for quick querying and an external API for image processing if needed.
3. **Order Service**: Manages orders from creation to completion. It handles order validation, tracks the status, and manages order history. This service may interact with the **Inventory Service** and **Payment Service**.
4. **Inventory Service**: Tracks stock levels, updates inventory when products are purchased, and notifies the Order Service if an item is out of stock.
5. **Payment Service**: Handles payment processing and transactions. It integrates with external payment gateways like Stripe or PayPal to manage payments securely.
6. **Recommendation Service**: Provides personalized product recommendations to users. This service could use machine learning models and data from the Order and User Services to suggest products based on user behavior and purchase history.
7. **Notification Service**: Sends email or SMS notifications to users for various events, such as order confirmation, shipment tracking, and payment confirmation.
8. **API Gateway**: Acts as a single entry point for client applications (web or mobile). The gateway routes requests to the appropriate microservices and provides load balancing, authentication, and throttling.

■ Implementation Steps

Step 1: Develop Each Microservice Independently

Each service can be built using a different technology stack if necessary. For example:

- **User Service**: Node.js with MongoDB
- **Order Service**: Java with PostgreSQL
- **Inventory Service**: Python with MySQL
- **Payment Service**: Python with integration to third-party payment APIs

Each microservice has its own codebase, database, and deployment pipeline, making it possible to update or scale each service independently.

Step 2: Define Communication Protocols

- **Synchronous Communication**: Services like the Order Service and Inventory Service communicate via REST or gRPC calls for real-time responses.
- **Asynchronous Communication**: For events like order completion or low inventory, the platform can use a message broker (e.g., Kafka, RabbitMQ) to send event notifications. This approach allows for decoupled services that don't need to know about each other's implementations.

Step 3: Implement API Gateway

- An API Gateway sits in front of all the services and provides a single entry point for all client applications (web, mobile, etc.).
- It handles requests from clients and routes them to the appropriate microservice, abstracts the complexity of managing multiple services, and enables centralized authentication and authorization.

Step 4: Ensure Scalability and Resilience

- **Horizontal Scaling**: Each service can be scaled horizontally by adding more instances to handle increased load.
- **Service Discovery**: Use a service discovery tool (e.g., Consul, Eureka) to keep track of running instances of each service and load balancing.
- **Circuit Breakers and Fallbacks**: Implement circuit breakers (e.g., with Hystrix) to prevent cascading failures in case a service goes down and set up fallbacks where necessary.

Step 5: Implement Monitoring and Logging

- Use centralized logging (e.g., ELK stack) and monitoring tools (e.g., Prometheus, Grafana) to keep track of the health of each service.
- Set up alerts for critical errors and anomalies to ensure quick recovery and minimal downtime.

■ Example Flow of an Order Process

1. **User places an order** on the front-end app, which sends a request to the **Order Service** through the **API Gateway**.
2. **Order Service** checks product availability by calling the **Inventory Service**.
3. If items are in stock, the **Order Service** proceeds to request payment from the **Payment Service**.
4. Once payment is successful, the **Inventory Service** updates stock, and the **Notification Service** sends a confirmation message to the user.
5. The **Order Service** marks the order as completed and may send a message to the **Recommendation Service** to update user preferences based on the purchased items.

■ Benefits of the Microservices Architecture for This Application

- **Independent Scaling**: Services like Product Catalog and Order Service can be scaled based on demand.
- **Fault Isolation**: If the Recommendation Service fails, it doesn't affect critical functionalities like order processing.
- **Technological Flexibility**: Different services can use the best-suited technologies for their specific tasks.
- **Faster Deployment**: Teams can deploy updates to individual services without affecting the whole system.

This architecture makes the e-commerce platform more flexible, scalable, and resilient, ensuring it can handle high traffic while remaining modular and easier to maintain over time.

2. Layered Architecture Style

■ Scenario: Building a Library Management System

Let's say a team needs to develop a **Library Management System** for a local library. The system should allow librarians to manage books, members, and loans. It should also enable library members to browse the catalog and borrow or return books.

The team decides to use a layered architecture for its simplicity and well-organized structure, which aligns well with the requirements of a Library Management System.

■ Layered Architecture Structure

The application can be divided into the following four layers:

1. **Presentation Layer**: This layer is responsible for displaying information to the user and handling user inputs. It typically includes the UI (web or mobile interface) and may contain controllers that manage user interactions.
2. **Application Layer (Service Layer)**: This layer contains the business logic, orchestrating the operations required by the application. It performs tasks such as validating input data, handling transactions, and coordinating responses to user actions.
3. **Domain Layer (Business Logic Layer)**: This layer defines core business rules and models that represent the library system's core entities, such as Book, Member, and Loan. It includes all the rules for how books are borrowed, returned, and managed.
4. **Data Access Layer**: This layer handles all interactions with the database. It includes methods for CRUD operations (create, read, update, delete) and abstracts database access, ensuring that other layers don't need to know the specifics of the database technology used.

■ Implementation Example

Here's how each layer might work together in the Library Management System:

1. Presentation Layer

- Provides a user interface (UI) for both librarians and library members.

- Contains forms and screens for actions like logging in, browsing the catalog, and managing loans.
- Sends requests to the Application Layer for actions like borrowing or returning a book.

2. Application Layer

- Receives requests from the Presentation Layer, validates them, and directs them to the appropriate business logic in the Domain Layer.
- Manages user authentication, ensuring only authenticated members or librarians can perform specific actions.
- Coordinates business workflows, such as handling the process for borrowing a book, including checking availability and managing loan records.

3. Domain Layer

- Contains business entities (Book, Member, Loan) and associated rules. For example:
  - A Book object has attributes like title, author, and ISBN.
  - A Loan object has attributes like borrowDate, dueDate, and status.
  - Business rules, such as "A book can only be borrowed if it's available" or "A loan is overdue if the current date is past the due date," are enforced here.
- Implements logic to update the status of books and manage loan details when actions are triggered.

4. Data Access Layer

- Contains methods to interact with the database, such as retrieving a list of books, checking if a book is available, and updating loan records.
- For example:
  - getBookById(id): Retrieves a book record based on its ID.
  - saveLoan(loan): Saves a new loan to the database.
- This layer abstracts the specifics of the database (e.g., SQL, NoSQL), so the rest of the application doesn't need to interact directly with it.

■ Example Flow of Borrowing a Book

1. **User Interaction**: A library member clicks the "Borrow" button on a book's detail page in the **Presentation Layer**.
2. **Application Layer**: The request to borrow a book is sent to the Application Layer, where:
   - It checks if the user is authenticated.
   - It calls the Domain Layer to verify if the book can be borrowed.
3. **Domain Layer**:
   - Checks the book's status to see if it's available.
   - If available, creates a new Loan record with the borrow date, due date, and sets the book's status to "borrowed."
4. **Data Access Layer**:
   - The new Loan record is saved to the database.

- The book's availability status is updated to "unavailable."

5. **Presentation Layer**: The member receives confirmation that the book has been borrowed, and the book's status is updated on their screen.

■ Benefits of Using Layered Architecture for This System

- **Separation of Concerns**: Each layer has a clear responsibility, making the codebase easier to understand and maintain.
- **Modularity**: Each layer can be developed and tested independently. Changes in the Data Access Layer (e.g., switching to a different database) do not affect the Presentation or Domain layers.
- **Reusability**: Business logic in the Domain Layer can be reused across multiple applications or interfaces (e.g., web, mobile).
- **Scalability**: The layered structure supports scalability, as each layer can be optimized independently. For instance, caching can be added at the Application Layer to improve performance.

This approach ensures a structured, organized system that is maintainable and easily extensible, fitting well for relatively straightforward applications like a Library Management System.

3. Pipeline Architecture Style

■ Scenario: Building a Data Processing Pipeline for an Image Processing Application

Imagine a team is developing an image processing application that applies a series of transformations to uploaded images. Each transformation (e.g., resizing, filtering, watermarking, and compression) must occur in a specific sequence, making this a perfect fit for the Pipeline architecture.

■ Architecture of the Pipeline

In this architecture, the application is divided into independent processing components, each performing a specific task. Data flows sequentially through each component.

Example Pipeline Stages

1. **Input Stage**: Receives and validates the uploaded image file.
2. **Resize Stage**: Resizes the image to the required dimensions.
3. **Filter Stage**: Applies color filters or adjustments, like grayscale or sepia tone.
4. **Watermark Stage**: Adds a watermark to the image if needed.
5. **Compression Stage**: Compresses the image for optimal file size.
6. **Output Stage**: Saves the final image and sends it to the user or stores it for future use.

■ Implementation Example

Each stage in the pipeline performs a transformation on the image, and they are linked together in a specific sequence. Here's how each stage might work:

1. **Input Stage**
   - Accepts the image file upload from the user interface.
   - Validates the file format (e.g., JPEG, PNG) and checks size restrictions.
   - Passes the validated image data to the next stage in the pipeline (Resize Stage).
2. **Resize Stage**
   - Resizes the image based on user specifications or default dimensions.
   - After resizing, it passes the modified image to the Filter Stage.
3. **Filter Stage**
   - Applies any chosen filter (e.g., grayscale, sepia, brightness adjustment).
   - Multiple filters can be applied as separate, sequential stages if needed.
   - Passes the filtered image to the Watermark Stage.
4. **Watermark Stage**
   - Adds a watermark (e.g., library logo or user-defined text) to the image.
   - Ensures the watermark is placed and sized appropriately.
   - Passes the watermarked image to the Compression Stage.
5. **Compression Stage**
   - Compresses the image to reduce file size while maintaining quality.
   - The compression algorithm may vary based on the file format and desired quality.
   - Passes the compressed image to the Output Stage.
6. **Output Stage**
   - Saves the processed image to a specified storage location (e.g., database, file system, cloud storage).
   - Generates a URL for the processed image if it's stored online.
   - Returns the processed image to the user, allowing them to download it or view it directly.

■ Example Flow of Processing an Image

1. **User Uploads Image**: The user uploads an image, which enters the pipeline at the **Input Stage**.
2. **Image Resizing**: The **Resize Stage** resizes the image according to the specified dimensions.
3. **Applying Filter**: The **Filter Stage** applies the desired color adjustment to the resized image.
4. **Adding Watermark**: The **Watermark Stage** places a watermark on the filtered image.
5. **Compressing Image**: The **Compression Stage** compresses the watermarked image to reduce file size.
6. **Output**: The **Output Stage** saves the final processed image and provides a download link or preview to the user.

■ Benefits of Pipeline Architecture for This Application

- **Modularity**: Each stage in the pipeline is a standalone unit, making it easy to add, remove, or modify individual stages without impacting others.

- **Scalability**: Each stage can be scaled independently, allowing the application to handle more users by parallelizing the processing steps.
- **Reusability**: Each stage can be reused in different contexts. For example, the Filter Stage can be reused for other applications requiring image color adjustments.
- **Maintainability**: The clear separation of concerns means each stage can be tested and debugged independently, making the system easier to maintain.

■ Variations and Enhancements

- **Parallel Processing**: If certain stages are independent of others (e.g., adding multiple filters), they can be processed in parallel to improve efficiency.
- **Error Handling**: Error handling can be added to each stage to manage failures. For instance, if the Watermark Stage fails, the pipeline can revert to the previous stage or retry the operation.
- **Dynamic Pipeline Creation**: The pipeline could dynamically configure stages based on user preferences. For instance, if a user only wants resizing and compression, the pipeline can skip unnecessary stages.

■ Example Use Cases of Pipeline Architecture

- **Data Transformation Pipelines**: Converting, cleaning, and transforming data for analytics or machine learning applications.
- **Video Processing**: Applying sequential transformations such as encoding, watermarking, compression, and storage.
- **Financial Data Processing**: Sequentially validating, cleaning, analyzing, and archiving transaction records in banking applications.

This Pipeline architecture allows the image processing application to operate efficiently, with clear and well-organized stages that can be easily modified, scaled, or reused as needed.

4. Microkernel Architecture Style

■ Scenario: Building a Text Editor with Plug-in Support

Imagine a team is developing a **text editor** application. They want the editor to have a core set of features (like basic text editing) while allowing users to add advanced functionalities, such as spell check, grammar suggestions, syntax highlighting, and more. The Microkernel architecture is an excellent fit for this application, as it allows for a lightweight core with optional, easily installable plug-ins.

■ Architecture of the Text Editor

1. **Microkernel (Core)**: Provides the essential features of the text editor, such as opening, editing, saving, and basic formatting of text. This core is stable, lightweight, and performs only the most fundamental tasks required by the application.
2. **Plug-ins (Modules)**: Add extra features that users can install or activate based on their needs. Each plug-in communicates with the microkernel and extends the editor's functionality without changing the core.

3. **API for Plug-ins**: Defines how plug-ins can interact with the core. The API provides access to the editor's main functions, such as file access, text manipulation, and UI integration, allowing plug-ins to add new capabilities seamlessly.

■ Example Plug-ins for the Text Editor

1. **Spell Checker Plug-in**: Checks the text for spelling errors, underlining incorrect words and suggesting corrections.
2. **Grammar Checker Plug-in**: Analyzes the text for grammatical errors and provides suggestions for improvement.
3. **Syntax Highlighting Plug-in**: Adds syntax coloring for specific programming languages (e.g., JavaScript, Python).
4. **Word Count Plug-in**: Tracks and displays word and character counts in real time.
5. **Export to PDF Plug-in**: Adds functionality to export the document as a PDF file.
6. **Auto-save Plug-in**: Automatically saves the document at regular intervals or whenever changes are detected.

■ Implementation Example

In this example, the Microkernel architecture allows the team to design a basic text editor with core functionality while offering extensibility through plug-ins.

1. Microkernel (Core)

- Provides essential text editing features: creating, opening, editing, and saving files.
- Manages a simple UI that displays the text and basic formatting options.
- Defines a **plug-in management system** to load and initialize plug-ins.
- Publishes an **API** that exposes functions (e.g., text manipulation, UI update hooks) that plug-ins can call.

2. Plug-ins

Each plug-in adds a distinct feature and is developed independently of the core. Here’s how some plug-ins might interact with the microkernel:

- **Spell Checker Plug-in**:
  - Uses the core API to access the current text.
  - Analyzes the text for misspelled words.
  - Uses the core API to underline misspelled words in the editor and display suggestions when a user right-clicks on an underlined word.
- **Syntax Highlighting Plug-in**:
  - Detects the file type (e.g., .js or .py) and applies syntax highlighting based on that.
  - Uses the core API to format text with different colors based on language-specific keywords.
- **Export to PDF Plug-in**:
  - Provides a new menu item, "Export as PDF," in the editor’s UI.

- When selected, converts the document to PDF format and prompts the user to save it.
- Uses the core API to retrieve document contents and format.

3. API for Plug-ins

- The API provides plug-ins with functions to interact with the editor's text and UI. Common functions in the API might include:
  - getText(): Retrieves the text content from the editor.
  - setTextFormatting(position, style): Applies formatting (e.g., underline, color) to specific text positions.
  - addMenuItem(label, callback): Allows plug-ins to add menu options to the editor's UI.
  - onSave(callback): Registers an event that triggers when the document is saved.

■ Example Flow of Adding Spell Check Functionality

1. **Plug-in Initialization**: The Spell Checker plug-in is installed and initialized by the microkernel when the editor starts.
2. **Text Access via API**: The Spell Checker uses the getText() API function to access the document's content.
3. **Error Detection**: The plug-in scans the text for spelling errors.
4. **Formatting via API**: For each misspelled word, the plug-in uses the setTextFormatting() API to underline it in red.
5. **User Interaction**: When the user right-clicks a misspelled word, the plug-in displays suggestions, allowing the user to replace the word with a correct one.

■ Benefits of Microkernel Architecture for This Application

- **Extensibility**: New plug-ins can be added without modifying the core, enabling users to customize the editor's functionality.
- **Maintainability**: The core is lightweight and stable, with plug-ins adding complexity only as needed.
- **Scalability**: Plug-ins can be independently developed, tested, and updated without affecting the core system.
- **Adaptability**: Users can enable or disable plug-ins based on their needs, making the application highly adaptable for different user requirements.

■ Additional Use Cases of Microkernel Architecture

- **Operating Systems**: Many OS kernels use the microkernel approach, where core functionalities (e.g., memory and process management) are part of the kernel, and additional features are added through modules.
- **Web Servers**: Some web servers have a core request-handling mechanism, with additional modules for handling specific protocols or features.
- **Business Applications**: ERP systems often have a core module for accounting, with additional plug-ins for inventory, HR, and CRM functionalities.

The Microkernel architecture allows this text editor application to remain lightweight and highly customizable, providing a robust core while allowing advanced features to be added as needed. This enables the application to cater to a wide range of users, from those needing basic editing tools to power users who require advanced functionalities.

5. Service-Based Architecture Style

■ Scenario: Developing an E-Commerce Platform

Imagine a team is building an **e-commerce platform**. The platform needs to handle various functionalities, including user management, product catalog, inventory, order processing, and payments. The team decides to use a **Service-Based Architecture** because it allows modularity and easy scalability without the complexity of fully independent microservices.

■ Architecture of the E-Commerce Platform

1. **Core Services**: Each core service handles a specific area of functionality and communicates with others through a shared API gateway or direct calls.
2. **API Gateway**: Acts as a unified entry point for external requests, routing them to the appropriate services.
3. **Shared Database**: All services share a common database, which simplifies data consistency and reduces complexity compared to each service having its own database.
4. **Communication**: Services communicate through well-defined interfaces, often over HTTP or lightweight messaging.

■ Example Services in the E-Commerce Platform

1. **User Service**: Manages user accounts, authentication, and user profiles.
2. **Product Catalog Service**: Handles the management and display of products, including information like pricing, descriptions, and categories.
3. **Inventory Service**: Manages stock levels and tracks inventory for each product.
4. **Order Service**: Handles order creation, tracking, and management of order status.
5. **Payment Service**: Processes payments, integrates with third-party payment gateways, and updates payment status.

■ Implementation Example

Here's how these services work together in a Service-Based Architecture:

1. User Service

- Manages all user-related information, including account details, login, and password management.
- Exposes endpoints for creating accounts, logging in, and updating profiles.
- Uses the shared database to store user information and retrieve it as needed.

2. Product Catalog Service

- Stores and retrieves information about available products, such as descriptions, prices, images, and categories.
- Provides endpoints for browsing products, searching by category, and viewing product details.
- Directly communicates with the Inventory Service when displaying stock availability.

3. Inventory Service

- Tracks stock levels for each product, ensuring that customers can only purchase available items.
- Provides an API for checking and updating inventory levels.
- Can receive notifications from the Order Service to adjust inventory levels when a purchase is completed.

4. Order Service

- Handles order creation, updates order status, and tracks order history for users.
- When a user places an order, the Order Service:
  - Communicates with the Inventory Service to ensure the items are in stock.
  - Interacts with the Payment Service to process the payment.
- If successful, the order is confirmed, and the Inventory Service is updated.

5. Payment Service

- Manages payment processing by integrating with third-party payment gateways.
- Exposes endpoints for initiating, processing, and updating payment status.
- Notifies the Order Service once payment is confirmed, allowing it to finalize the order.

6. API Gateway

- Serves as a single point of entry for clients (web or mobile), routing requests to the appropriate services.
- Simplifies the communication for external clients, reducing the need for them to interact directly with multiple services.
- Provides additional capabilities like rate limiting, request logging, and load balancing.

■ Example Flow: Placing an Order

1. **User Browses Products**: The user browses the catalog via the **Product Catalog Service** through the API Gateway. Each product displays its current stock, checked by the **Inventory Service**.
2. **User Places Order**: The user adds items to their cart and places an order.
3. **Order Creation**:
   - The **Order Service** receives the request to create an order.
   - It calls the **Inventory Service** to ensure the items are available.

4. **Payment Processing**:
    - The **Order Service** initiates payment by sending a request to the **Payment Service**.
    - The Payment Service processes the payment with a third-party gateway and updates the payment status.
5. **Order Confirmation**:
    - If payment is successful, the **Order Service** confirms the order and updates the order status.
    - The Inventory Service reduces the stock levels for the purchased items.
6. **Notification**: The user receives a confirmation notification with order details.

■ Benefits of Service-Based Architecture for This Application

- **Modularity**: Each service can be developed, tested, and deployed independently, improving development speed and quality.
- **Scalability**: Each service can be scaled separately based on demand. For example, the Order Service can be scaled up during peak shopping periods.
- **Ease of Maintenance**: Since each service has a specific responsibility, the system is easier to debug and maintain. Bugs can be isolated to individual services without affecting the entire application.
- **Simplicity Over Microservices**: Unlike a full microservices architecture, a Service-Based Architecture doesn't require each service to have its own database, reducing the complexity of maintaining data consistency.

■ Additional Use Cases of Service-Based Architecture

- **Banking Applications**: Services can be created for account management, transaction processing, customer support, and reporting.
- **Healthcare Systems**: Services might handle patient records, appointment scheduling, billing, and prescription management.
- **Education Platforms**: Separate services can manage user accounts, course content, grading, and messaging between students and instructors.

The Service-Based Architecture allows the e-commerce platform to be modular and flexible, making it possible to add or update services as needed without disrupting the entire application. By sharing a database, it simplifies data management while retaining many of the benefits of a more complex microservices architecture.

6. Event-Driven Architecture Style

■ Scenario: Building a Real-Time Order Processing System for an E-Commerce Platform

Imagine a team is building a real-time **order processing system** for an e-commerce platform. When a user places an order, multiple actions need to happen: inventory needs to be updated, payment processed, order confirmation sent, and shipping arranged. Since each of these steps is independent and can happen concurrently, an **Event-Driven Architecture** is ideal for this system.

■ Architecture of the Real-Time Order Processing System

1. **Event Producers**: Components that generate events. In this case, the main event producer is the **Order Service** when an order is placed.
2. **Event Consumers**: Services that listen for events and perform actions in response. Examples include the **Inventory Service**, **Payment Service**, **Shipping Service**, and **Notification Service**.
3. **Event Bus (Message Broker)**: A central hub that routes events from producers to consumers. This can be implemented using a message broker (e.g., Kafka, RabbitMQ, or Amazon SNS) to decouple components and ensure reliable event delivery.
4. **Event Store**: A storage solution where events can be stored for auditing, replay, and analytics.

■ Example Events in the Order Processing System

1. **Order Placed**: Triggered when a user places an order, initiating the entire process.
2. **Inventory Updated**: Triggered after the inventory is adjusted based on the ordered items.
3. **Payment Processed**: Emitted when the payment has been successfully processed.
4. **Order Confirmed**: Sent once the order has been confirmed and is ready for shipping.
5. **Shipping Scheduled**: Emitted when the shipping details have been finalized.
6. **Notification Sent**: Triggered to inform the user of each major update in the order process.

■ Implementation Example

Here's how the Event-Driven Architecture could be applied in this order processing system:

1. Order Placed Event

- **Producer**: The **Order Service** is the event producer, sending an "Order Placed" event when a user completes an order.
- **Event Details**: Contains details such as order ID, user information, and list of items.

2. Inventory Service (Event Consumer)

- **Action**: Listens for the "Order Placed" event.
- **Response**: Adjusts the inventory based on the items in the order.
- **Produces**: Once inventory is updated, the Inventory Service emits an "Inventory Updated" event.

3. Payment Service (Event Consumer)

- **Action**: Also listens for the "Order Placed" event.
- **Response**: Initiates the payment process with a third-party payment gateway.
- **Produces**: If payment is successful, it emits a "Payment Processed" event; if unsuccessful, it emits a "Payment Failed" event.

4. Order Service (Intermediate Event Consumer)

- **Action**: Listens for the "Payment Processed" and "Inventory Updated" events.
- **Response**: Once both payment and inventory are confirmed, the order is marked as confirmed.
- **Produces**: Emits an "Order Confirmed" event.

5. Shipping Service (Event Consumer)

- **Action**: Listens for the "Order Confirmed" event.
- **Response**: Schedules shipping and prepares the order for dispatch.
- **Produces**: Emits a "Shipping Scheduled" event with details of the shipping provider, tracking number, and estimated delivery date.

6. Notification Service (Event Consumer)

- **Action**: Listens for events like "Order Placed," "Payment Processed," "Order Confirmed," and "Shipping Scheduled."
- **Response**: Sends real-time notifications (e.g., emails, SMS) to the user with updates on each stage of the order.
- **Produces**: Emits a "Notification Sent" event after each notification is delivered.

■ Example Flow: Processing an Order in Real-Time

1. **Order Placed**: The user places an order. The **Order Service** emits an "Order Placed" event, which the system broadcasts via the **Event Bus**.
2. **Inventory and Payment Processing**:
    - **Inventory Service** receives the "Order Placed" event, adjusts stock levels, and emits an "Inventory Updated" event.
    - **Payment Service** also receives the "Order Placed" event, processes the payment, and emits a "Payment Processed" event if successful.
3. **Order Confirmation**: The **Order Service** waits for both "Inventory Updated" and "Payment Processed" events. Once received, it emits an "Order Confirmed" event.
4. **Shipping Scheduling**: The **Shipping Service** receives the "Order Confirmed" event, schedules the shipment, and emits a "Shipping Scheduled" event.
5. **User Notifications**: The **Notification Service** listens to all events related to the order and sends the user updates at each stage, like "Order Placed," "Order Confirmed," and "Shipping Scheduled."

■ Benefits of Event-Driven Architecture for This Application

- **Real-Time Processing**: EDA enables immediate reaction to events, allowing the system to provide real-time updates and process tasks as soon as the events occur.
- **Scalability**: Each service can scale independently. For example, if payment processing becomes a bottleneck, only the Payment Service needs to be scaled.
- **Loose Coupling**: Services don't depend directly on each other; they only need to know what events to listen to. This reduces interdependencies, making the system more resilient and flexible.
- **Extensibility**: New features or services can be added by simply introducing new events or consumers without impacting existing components. For instance, adding a new "Discount Service" to process promotional codes would not affect the core system.

■ Additional Use Cases of Event-Driven Architecture

- **Financial Services**: In banking, EDA can trigger fraud detection, alerts, and reporting whenever suspicious transactions occur.
- **IoT Systems**: IoT applications often use EDA to react to sensor data in real-time, like temperature changes triggering alarms or automated responses.
- **Gaming**: Real-time multiplayer games use EDA to handle events like player actions, updates, and game state changes across multiple devices.
- **Social Media**: EDA enables social media platforms to update notifications, news feeds, and user engagement data instantly when users interact with posts.

This Event-Driven Architecture allows the order processing system to be highly responsive, flexible, and scalable, providing an ideal foundation for real-time, asynchronous communication among services.

7. Space-Based Architecture Style

■ Scenario: Building a High-Traffic E-Commerce Platform

Imagine a team is developing a high-traffic **e-commerce platform** for a popular online sale event, such as Black Friday. During peak times, the platform will face intense demand, with thousands of concurrent users browsing products, placing orders, and updating inventory. The team chooses **Space-Based Architecture** to ensure the application can scale seamlessly under heavy loads without compromising speed or reliability.

■ Architecture of the E-Commerce Platform

1. **Processing Unit (PU)**: Each processing unit contains all necessary application components, including business logic and in-memory data storage, to process a subset of user requests independently.
2. **Virtualized Middleware (Data Grid)**: A distributed in-memory data grid is used as a shared, virtualized storage layer across all processing units, enabling fast data access without a centralized database.

3. **Data Replication and Synchronization**: Data is replicated across multiple nodes, ensuring that the system remains consistent and available even if a node fails.
4. **Load Balancer**: Distributes user requests across the available processing units to balance the load dynamically.
5. **Elastic Scalability**: Nodes can be added or removed based on load, allowing the system to scale horizontally.

■ Key Components

1. **User Service**: Handles user-related tasks, such as authentication, user profile management, and shopping cart management.
2. **Product Service**: Manages product catalog information, including product details, pricing, and inventory.
3. **Order Service**: Manages the order lifecycle, including placing, processing, and updating order statuses.
4. **Inventory Service**: Tracks inventory levels and updates stock in real-time as orders are placed.

■ Implementation Example

In a Space-Based Architecture, each processing unit contains a copy of the required services (User, Product, Order, Inventory) and in-memory data needed to handle requests, ensuring high availability and fast access times.

1. Processing Units

- Each processing unit operates independently and handles its share of requests.
- For example, a processing unit may manage user requests to view products, add items to the cart, and place orders, all in-memory.
- Each unit has a subset of application data stored locally, which is replicated across other processing units for consistency.

2. Data Grid

- Instead of a centralized database, data is stored in a distributed in-memory data grid.
- This data grid acts as a shared storage layer that synchronizes data between processing units.
- Data changes made in one processing unit are propagated across the data grid to maintain consistency.

3. Data Replication and Synchronization

- Data for user sessions, shopping carts, and inventory levels are replicated across processing units.
- If a processing unit goes down, its data is still available in other units, enabling high availability.
- Data replication and synchronization keep the application state consistent and fault-tolerant.

4. Load Balancer

- A load balancer distributes incoming requests to the processing units based on current load.
- During peak times, additional processing units are added, and the load balancer directs traffic accordingly.
- When demand decreases, the load balancer reduces the number of active processing units to optimize resource usage.

■ Example Flow: Placing an Order

1. **User Browses Products**: A user browses products, and the request is routed by the load balancer to an available processing unit.
    - The **Product Service** in the processing unit retrieves product information from the data grid or local cache, displaying products to the user.
2. **User Adds Item to Cart**: The user adds an item to their cart.
    - The **User Service** in the processing unit updates the cart data in memory.
    - This data is synchronized across other processing units to ensure that if the user's session moves to another unit, their cart is intact.
3. **Order Placement**:
    - The user places an order, and the **Order Service** in the processing unit processes it.
    - The **Inventory Service** updates stock levels immediately in the local memory and synchronizes with the data grid, ensuring consistency across units.
    - The **Order Service** confirms the order, saving it in memory and replicating it across other units.
4. **Real-Time Inventory Updates**:
    - As stock levels change with each order, these updates are propagated across processing units via the data grid.
    - Users and services receive real-time availability updates, preventing overselling of products during peak times.

■ Benefits of Space-Based Architecture for This Application

- **High Scalability**: By distributing data and load across multiple processing units, the platform can handle thousands of concurrent users.
- **Fault Tolerance**: If one processing unit fails, other units continue serving requests without data loss, thanks to data replication across the grid.
- **Real-Time Processing**: Since data is stored in memory, requests are processed in near real-time, delivering quick responses for actions like adding items to the cart or placing an order.
- **Elasticity**: Processing units can be added or removed dynamically based on the application's needs, allowing the system to scale up during high-traffic events and scale down afterward.
- **Simplified Data Management**: By storing data in an in-memory grid rather than a traditional database, the application reduces the need for complex database management and caching layers.

■ Additional Use Cases of Space-Based Architecture

- **Gaming Applications**: Multiplayer online games that require real-time data sharing between players benefit from Space-Based Architecture for handling game state and player actions.
- **Social Media Platforms**: High-traffic platforms that need to process user-generated content, comments, and likes in real time can use this architecture to handle large volumes of interactions.
- **Financial Services**: Applications like stock trading platforms that require fast data processing and low-latency transactions can leverage space-based architecture for quick access to trade data and real-time updates.

Space-Based Architecture enables the e-commerce platform to handle high traffic and real-time processing demands efficiently. It's an ideal choice for applications that anticipate significant spikes in usage, require rapid data access, and need to maintain consistent performance under heavy load.